

DATE: 1/1/20

Chapter 10:-

Association Analysis

⇒ Machine learning methods in data mining:

Parametric Method:

⇒ Assume a specific form for the model and use a fixed number of parameters.

⇒ used for:

- Density estimation
- Classification
- Regression

⇒ models in this case are global models.

Pros:

- reduces problem of solving modeling with only a small number of parameters.

Cons:

Initial assumptions do not hold in many real world problems.

Non-parametric method:

⇒ Do not assume any specific form for the model and do not have a fixed number of parameters.

⇒ Assume that similar inputs have similar outputs.

⇒ Not assume any prior density or parametric form.

⇒ No single global model.

Note:

The discovery of association rules is one of the major technique in data mining.

Standard methodologies:

- Apriori techniques

↓
Extensions

FP Tree ← → CMAR

DATE: ___/___/20__

MTWTFSS

Association Analysis

⇒ Association rules analysis is a technique to uncover (mine) how items are associated to each other.

⇒ Such uncovered association between items are called association rules.

- checks dependency of ^{one} data items on another data items so map

Item set

⇒ A collection of items, $\{I_1, I_2, \dots, I_m\}$ is an item set. An itemset containing k items is called k -itemset.

Transaction

⇒ Database D consist of transaction, each containing a set of items.

Frequent patterns:

⇒ Patterns that appear frequently in a data set.

DATE: ___/___/20__

MTWTFSS

(i) Market Basket Analysis (ARM)

⇒ A typical example of frequent pattern mining for associations rules.

Defⁿ:

A collection of items bought by a customer in a single transaction.

ex:

A visit to a grocery store or an online purchase.

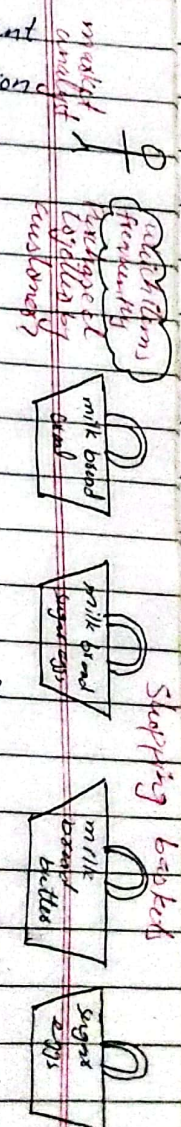
Data collection:

Retailer activity:

Retailers collect huge amounts of transaction data over time.

Purpose:

To analyze pattern in these transaction.



Market Basket Analysis

o Data mining technique used by retailers to increase sales by better understanding customer purchasing patterns.

o Analyze purchases that commonly happen together. Identifies customer buying habits by finding associations between the different items that customers place in their "shopping baskets".

Helpful for retailers to develop marketing strategies by gaining insight into which items are frequently bought together by customers.

ex:

People who buy shampoo might also buy conditioners

⇒ Applicable to retail industry.

DATE: 1/1/20

MTWTFSS

⇒ Market basket analysis analyzes customer buying habits by finding associations between the different items that customers place in their shopping baskets.

Finding Itemsets:

Def: Sets of items that often appear together in many transactions called itemset.

Usage:

Helps in improving store layouts, catalog designs, and web page arrangements.

Containment:

⇒ A transaction T contains a set of items A if A is a subset of T .

Association rule:

- o An implication in the form $A \rightarrow B$ if $\rightarrow$ then
- o A & B must be non-empty & disjoint.

DATE: 1/1/20

MTWTFSS

Support: (S) - how frequently an item appears in the dataset

⇒ Percentage of transactions that contain both A and B .

⇒ This is the probability of association $P(A \cup B)$. form: $\text{support}(A \rightarrow B) = P(A \cup B)$

Confidence (C) - how often two items occur together in dataset when the occurrence of A is given

⇒ Percentage of transactions containing A that also contain B .

⇒ This is the conditional probability. $P(B|A)$

formula: $\text{Confidence}(A \rightarrow B) = P(B|A) = \frac{\text{Frequency}(\text{support count})}{P(A)}$

The number of transactions that contain the itemset.

Calculating confidence:

Also known as absolute support.

market basket:

- o Set of items in a transaction

Strong Rules:

Rules with high confidence and strong support.

Parameters:

- ⇒ Finding all items that appear frequently in transaction. (min support count) →
- ⇒ Finding strong association among frequent items.

Problems:

- ⇒ Finding strong association among frequent items.
- ⇒ Levels of frequency of appearance determination

Functions:

- ⇒ Finding items that have significant impact on business.
- ⇒ Collecting info from numerous transactions
- ⇒ Generating rules from counts in trans.

Strengths:

Easy interpretations
Easy to start
flexible data formats
simplicity

Problem with mining association rules:

⇒ Decomposed into two phases:

0 Discover the large itemsets, that is, the sets of items that have transaction support s above a predetermined minimum threshold.

0 Use the large itemsets to generate the association rules for the database that have confidence c above a predetermined minimum threshold.

Solution:

Apriori algorithm

DATE: ___/___/20

(ii) Apriori Algorithm

- Use "bottom up" approach.

⇒ Proposed by R. Agrawal and R. Srikant in 1994.

⇒ It follows level-wise search approach where k items are utilized to explore $(k+1)$ items.

⇒ The algorithm leverages prior knowledge of frequent itemsets properties to reduce the search space and improve efficiency.

Purpose:

⇒ This algorithm is used to compute frequent itemsets in a database through iterations. mainly used for market basket analysis & helps to find those products bought together

Iteration:

⇒ Each iteration finds itemsets with a specific number of items.

Iteration process:

⇒ Each iteration has two steps:

1. candidate generation
2. candidate counting & selection

Components of Apriori:

(i) support = $\frac{\text{Support}(X)}{N}$

(ii) confidence = $\frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$

First Iteration:

Initial candidate:

In the first iteration, the candidates are all single items in the database.

Support counting:

The algorithm counts how often each single item appears in the database.

Selection:

Only the 1-itemset that has support above a specified threshold are kept as frequent itemsets.

Generating 2-itemset candidate:

⇒ To generate 2-itemset candidates in the second iteration, the apriori algorithm uses the cartesian product of frequent itemsets ($L_1 \times L_2$)

⇒ This operation results in pairs of items, forming candidate 2-itemsets.

DATE: 1/1/20
⇒ The number of candidate 2-itemsets are calculated based on the number of frequent 1-itemsets.

Pruning candidates:

⇒ Before counting the support of candidate itemsets, the algorithm prunes candidates that cannot be frequent based on the apriori property.

Apriori property:

⇒ The apriori property states that all non-empty subsets of frequent itemset must also be frequent.

⇒ This property is based on the observation that if an itemset is not frequent, then none of its supersets can be frequent.

⇒ Therefore, candidate itemsets with infrequent subsets are discarded prior to the counting step.

candidate 2-itemsets generation
the algorithm scans the database to count the support of each candidate.

⇒ Candidate 2-itemsets with support equal or greater than the minimum threshold are selected as frequent 2-itemsets.

Negative borders:

⇒ Apriori also counts the support of infrequent candidate itemsets that could not be pruned.

These infrequent itemsets, whose subsets are all frequent, constitute the negative borders.

⇒ The negative border is important for algorithm improvements and efficiency in generating large itemsets.

Algorithm termination:

⇒ When there are no more candidate itemsets to generate, indicating that all frequent itemsets have been identified.

DATE: 1/1/20

MTWTFSS

Brute force approach:

⇒ Analyzing all possible rules and filtering based on support and confidence.

Two-step approach:

⇒ Also two step-approach.

1. Generating frequent itemsets first.
2. Deriving association rules from them.

Advantages:

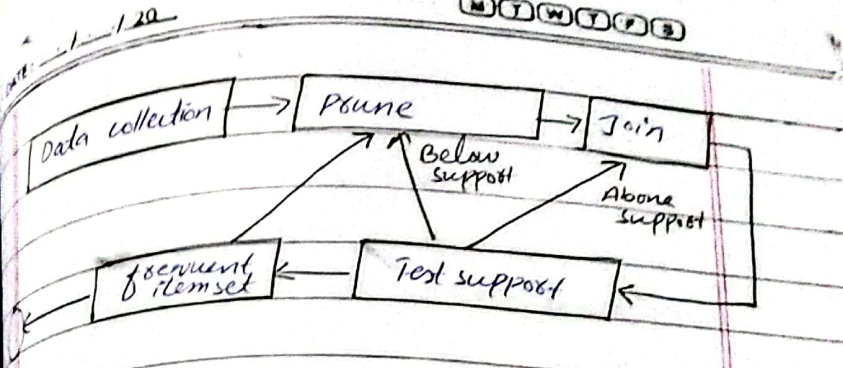
- Effective for deriving large itemsets. Easily parallelized
- Simple to understand.

Disadvantages:

- Computationally expensive
- Uses a uniform minimum support threshold. Requires many DB scans.

Versions: Three versions:

- Apriori faster in first iteration.
- Apriori^{id}
- Apriori Hybrid.



Example:

for the following given transaction data-set, generate rules using apriori algorithm. Consider the values as support = 50% and confidence = 75%.

Transaction ID	Items Purchased
01	Bread, cheese, egg, juice
02	Bread, cheese, juice
03	Bread, milk, yogurt
04	Bread, juice, milk
05	Cheese, juice, milk

DATE: ___/___/20

MTWTFSS

frequent itemset

items	frequency	support
Bread	4	$4/5 = 80\%$
cheese	3	$3/5 = 60\%$
egg	1	$1/5 = 20\%$
juice	4	$4/5 = 80\%$
milk	3	$3/5 = 60\%$
Yogurt	1	$1/5 = 20\%$

Prune

these

2-items

because their support is less than 50%.

make 2 items candidate set & write their frequency.

Item Pairs	freq	support
Bread, cheese	2	$2/5 = 40\%$
Bread, juice	3	$3/5 = 60\%$
Bread, milk	2	$2/5 = 40\%$
Cheese, juice	3	$3/5 = 60\%$
Cheese, milk	1	$1/5 = 20\%$
Juice, milk	2	$2/5 = 40\%$

These support

is 75%

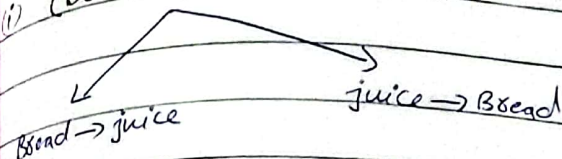
Association Rules

MTWTFSS

For Rules:

$\swarrow$ (Bread juice) $\searrow$ (Cheese juice)

(i) (Bread, juice)



$$\therefore \text{confidence } A \rightarrow B = \frac{\text{support}(A \cup B)}{\text{support } A}$$

(i) Bread $\rightarrow$ juice

$$= \frac{s(B \cup J)}{s(B)} = \frac{3}{5 \times 4/5} = \frac{3}{4} = 75\%$$

(ii) juice $\rightarrow$ Bread

$$= \frac{s(J \cup B)}{s(J)} = \frac{3}{5 \cdot \frac{4}{5}} = \frac{3}{4} = 75\%$$

 $\Rightarrow$ Both rules are valid.

$$\begin{aligned}
 &\text{(iii) Cheese} \rightarrow \text{juice} \quad \frac{s(\text{Cheese, juice})}{s(\text{Cheese})} \\
 &= \frac{3}{5 \cdot 3/5} = 100\%
 \end{aligned}$$

DATE: 1/1/20

(2) Juice $\rightarrow$ cheese:

$$= \frac{3}{5 \cdot \frac{4}{3}} = 75\%$$

Note: All the rules are good and valid.

(M)(T)(W)(T)(F)(S)

DATE: 1/1/20

Improving the efficiency of Apriori algorithm.

Basic apriori algorithm:

- Scans the database multiple times:

- Each scan is dependent on the size of the largest frequent itemset.

Enhancement to Apriori:

- Aim to reduce the number of database scan.

- Reduce the number of candidate itemsets counted in each iteration.

Various strategies:

i) Partition-based:

- Two scans are required.

- Divides the database into small, memory-sized partitions.

1st scan:

- o Compute locally frequent itemsets within each partition.

2nd scan:

- o Count the support of all locally frequent itemsets in the entire database to identify globally frequent itemsets.

2. Sampling algorithm:

Scan 1:

Take a sample, and generate candidate itemsets.

Scan 2:

Scan the database to count their exact support and the support of the negative border.

3) Incremental updating:

maintaining frequent itemsets:

update frequent itemsets without re-mining the whole database.

Reusing information:

Integrate old and new support information to reduce the number of candidates to examine.

mining at multiple

4) Concept level association:

- o Association might be more interesting at a higher level concept rather than detailed level.

- o mine frequent itemsets at generalized abstraction levels to capture these patterns.

5) Handling hierarchies in a Data:

- o utilizes is-a hierarchy where items have multiple abstraction levels.

- o Detect relationships involving higher hierarchy levels.

(ii) FB Growth Method

Problem with Apriori:

- Scalability issue occurs with the apriori algorithm.
- Apriori needs to generate and test huge number of candidate itemsets.
- For instance, for an itemset of length 100, would need about 10^{30} candidates, requiring many database scans and becoming very slow and complex.

Overview of FP method:

The frequent pattern growth was proposed by Jianwei Han, is a method of efficiently mining the complete set of frequent patterns without generating candidate items. It overcomes the limitations of apriori. Use compact representation of

Key concept

Frequent pattern Tree (FP-tree):

A compact data structure that stores quantitative information about frequent patterns in database. Done to mine all frequent patterns

Divide-and-conquer strategy:

The algorithm splits the problem into smaller parts, solves them individually, and combines the results.

Steps of FP-Growth

(i) Database Compression:

First Scan:

Count the occurrences of items to identify frequent items.

Sort items:

Arrange frequent items in descending order of their frequency.

Second Scan:

Construct the FP-tree by inserting transactions, maintaining the order of frequent items.

DATE: 1/1/20

MTWTFSS

(ii) FP-tree Construction:

Root node:

Initialize with a null root

Add transactions:

for each transaction, create a path in the tree. if the path shares a prefix with an existing path, increment the counts along the shared prefix.

Mining frequent patterns from FP-tree:

Conditional pattern base: for each item, construct its conditional pattern base.

Conditional FP-tree:

Create an FP-tree from each conditional pattern base.

Recursive mining:

Recursively mine each conditional FP-tree to find frequent patterns.

DATE: 1/1/20

MTWTFSS

Advantages:

⇒ Requires fewer database scans (only 2).
⇒ No need for candidate generation, which makes it faster and more efficient.

⇒ Efficient and scalable for both long and short frequent patterns.

Disadvantages:

⇒ Constructing the FP-tree can be complex and memory-intensive.

⇒ might not fit in memory for large databases.

Example#

Q. Generate FP tree for the following transaction dataset (minimum support = 30%)

DATE: 1/1/20

MTWTFSS

TID	Items
1	E, A, D, B
2	D, A, C, E, B
3	C, A, B, E
4	B, A, D
5	D
6	D, B
7	A, D, E
8	B, C

minimum no of transaction = $2 \cdot 4 = 3$

Items	frequency	Priority
A	5	3
B	6	1
C	3	5
D	6	2
E	4	4

Lower priority no means
higher priority.

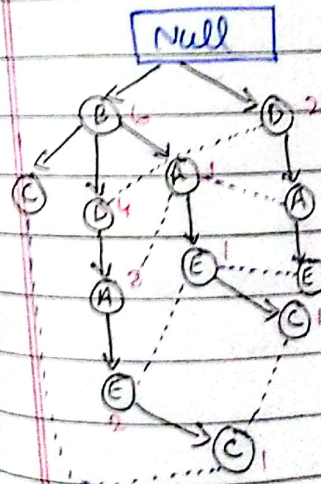
DATE: 1/1/20

MTWTFSS

Order the items according to
priority.

TID	Items	ordered items
1	F, A, D, B	B, D, A, E
2	D, A, C, E, B	B, D, A, E, C
3	C, A, B, E	B, A, E, C
4	B, A, D	B, D, A
5	D	D
6	D, B	B, D
7	A, D, E	D, A, E
8	B, C	B, C

Construct FP tree using ordered items:



B: 1, 2, 3, 4, 5, 6

D: 1, 2, 3, 4

A:

E: 1, 2, 3

C: 1, 2

A: 1

E: 1

C: 1

D: 1, 2

A: 1

C: 1

E: 1

FP VS Apriori

Apriori	FP Growth
Approach:	
Generate candidate itemsets.	Construct FP-tree and mines from it.
Database scans:	
multiple, one for each level.	Two: one for counting, one for FP-tree.
memory usage:	
Less efficient	more compact, but tree can be large.
search method:	
BFS	DFS
complexity:	
Simplex to implement.	more complex due to tree construction.

Associative-classification method
CMAR

CMAR (classification based on multiple association rules) is a method adapted from the FP-growth algorithm combining classification and association rule mining.

It generates association rules in the form $R:P \rightarrow C$ based on support and confidence thresholds.

These rules are used to classify new samples.

CMAR scans training data to find frequent itemsets, constructs an FP-tree, and generates classification rules.

Two Phases:

- (1) Rule generation or training
- (2) Classification or testing

Multidimensional Association

Rules:

- ⇒ Multidimensional transactional databases contain tuples with structured attributes and associated data itemsets.
- ⇒ mining involves two steps:
 - Discovering dimensional information patterns
 - finding frequent itemsets from the projected sub-database
- ⇒ MD patterns are found using a modified bottom up computation algorithm.
- ⇒ The algorithm iteratively refines patterns, starting with the first dimension and progressing to subsequent dimensions.
- ⇒ A processing tree visualize the search process and discovered MD patterns.

